

COMPANY OPERATING SYSTEM

Cloud Engineering & DevOps Service Division Playbook

From Zero to Professional: training, client delivery, security, reliability, pricing and sales

Developed by **V. LOKESHWARAN**
VEL Software Solutions
velsoftwaresolutions.in@gmail.com

Document VEL-CD-001 · Version 1.0 (draft for management approval)
Tool versions, limits and cloud prices change. Always verify against official provider documentation and pricing calculators before building or quoting.

How to Use This Playbook

This document is the single operating manual for the Cloud Engineering & DevOps division. It does two jobs: **(1)** it trains a beginner from zero to a professional who can join supervised client projects, and **(2)** it defines the mandatory lifecycle every client project follows, with approval gates, deliverables and acceptance criteria.

> **GOLDEN RULES**
>
> 1. Do not skip stages. Do not start a stage until the previous stage's acceptance criteria are met and its approval gate is signed.
> 2. “It runs” is not “it is production-ready.” Production-ready means secure, monitored, backed up, restorable, documented, cost-controlled and handed over.
> 3. Never run destructive commands (`rm -rf`, `terraform destroy`, database drops) without understanding the effect and having a verified backup.
> 4. Never put passwords, keys or tokens in Git, chat, documents or the company portal.
> 5. Never promise savings, uptime or security outcomes without evidence.
> 6. Every production change needs a second person's review.

Reading path for a beginner

Part A: Learn (L1–L4) → Part B: Cloud & tools → Part C: Project lifecycle (18 stages) → Part D: Business & sales → Part E: Practical projects & certification

Contents

1. Company service portfolio
2. Learning roadmap: Levels 1–4
3. Cloud platform engineering (AWS first)
4. DevOps toolchain
5. Client project lifecycle: Stages 01–18
6. Roles and project management
7. Internal operations portal
8. Pricing and profitability
9. Client acquisition and deal closing
10. Practical training projects and sign-off
11. Appendices: checklists, templates, glossary

1. Company Service Portfolio

Five commercially deliverable service lines. Each can be sold alone or combined.

Line	What we deliver	Typical client trigger
A. Cloud Infrastructure	Architecture design; AWS / Azure / GCP builds; servers and VMs; storage and databases; networking and firewalls; load balancers and auto-scaling; migration.	"Our server is a single box under someone's desk / hosting is slow, costly, fragile."
B. DevOps Engineering	Git/GitHub workflows; CI/CD; build and release automation; Docker; Kubernetes (only when justified); IaC; configuration management; automated test integration; rollback systems.	"Deployments are manual, risky and slow."
C. Cloud Security	IAM and least privilege; secrets management; network security; encryption; configuration reviews; vulnerability and dependency scanning; security monitoring; backup/DR; compliance readiness support.	"A customer or auditor asked about our security."
D. Monitoring & Reliability	Infrastructure and application monitoring; central logging; alerts; incident management; availability checks; capacity planning; performance tuning; DR testing.	"We find out about outages from customers."
E. Managed Cloud Services	Ongoing maintenance, patching, CI/CD upkeep, backup verification, cost optimisation, alert response, monthly reports, SLA-based support, upgrades.	"We do not have an ops team."

COMPLIANCE WORDING

We offer "compliance readiness support." We never certify a client as compliant and never claim a system is "secure" or "guaranteed." Certification is done by authorised auditors.

2. Learning Roadmap: Levels 1–4

For **every** topic the trainee must receive: definition → why it matters → real-world example → practical exercise → commands and expected output → common mistakes → a short assessment. The tables below give that pack for each topic. A level is passed only when all exercises are demonstrated to the Technical Lead.

Level 1 — Computer and IT Fundamentals (about 2 weeks)

Topic	Simple definition	Why it matters / example	Exercise & check
Hardware & software	Hardware is physical (CPU, RAM, disk); software is instructions that run on it.	A slow site may be a RAM shortage, not bad code.	Identify CPU, RAM, disk on your laptop. Explain which is exhausted when a video editor freezes.
Operating systems; Windows vs Linux	The OS manages hardware and programs. Linux runs most servers; Windows is common on desktops.	~Most cloud servers run Linux: cheaper, scriptable, no GUI needed.	List three differences (licensing, shell, file paths `C:\` vs `/`).
Files, folders, permissions	Who may read, write or execute each file.	Wrong permissions are a top cause of both breakages and breaches.	Create a file; make it readable only by you; prove another user cannot read it (Level 2).
Command line & terminal	Typing commands instead of clicking.	Automation is only possible with commands.	Open a terminal; run `pwd`, `ls`.
Processes & services	A process is a running program; a service is one that runs in the background.	Web servers and databases are services.	Find the process using most CPU.
IP addresses	The address of a device on a network (e.g. 192.168.1.10).	Servers are reached by IP; clients use names.	Find your private and public IP; explain the difference.
DNS	The phone book that turns `example.com` into an IP.	DNS mistakes make working sites “disappear.”	`nslookup example.com` / `dig example.com`; explain the answer.
HTTP/HTTPS, ports, protocols	HTTP is the web's language; HTTPS adds encryption. A port selects the service (80, 443, 22, 5432).	A firewall blocking 443 = “site down.”	`curl -I https://example.com`; name the status code.

Client-server; public/private networks; firewalls	Client asks, server answers. Firewalls allow or block traffic by rules.	Databases must live on private networks.	Draw client → internet → firewall → web server → DB.
Virtualisation; databases; app servers	One physical machine runs many virtual machines. A database stores data; an app server runs business logic.	Cloud VMs are virtualisation sold by the hour.	Install VirtualBox/WSL; run a Linux VM.

Common mistakes: confusing public with private IPs; assuming "HTTPS" means the app is secure; opening all ports "to make it work."
Assessment: 20-question quiz (pass ≥ 80%) + whiteboard the request path for a website.

Level 2 — Linux Administration (about 3 weeks)

Set up Ubuntu Server (VM first, then a cloud VM). Topics: install, SSH, users/groups, permissions, directories, packages, systemd, processes, disk/memory, logs, firewall, network troubleshooting, cron, shell scripting, hardening.

> **SAFETY BEFORE COMMANDS**
> Practise only on a disposable VM. Before any delete/overwrite: look first (`ls`), copy a backup, then act. Never run a command you cannot explain.

Command	What it does	Example → expected output	Risk
`pwd`	Shows current folder	`pwd` → `/home/trainee`	None
`ls -la`	Lists files, incl. hidden, with permissions	Rows like `-rw-r--r-- 1 trainee trainee 12 Jan 1 notes.txt`	None
`cd`	Changes folder	`cd /var/log`	None
`mkdir -p`	Creates folders	`mkdir -p lab/week1`	Low
`cp -r`	Copies	`cp notes.txt notes.bak`	Can overwrite
`mv`	Moves/renames	`mv a.txt b.txt`	Can overwrite
`rm`	Deletes. **No recycle bin.**	`rm notes.bak` . Never `rm -rf /` or on unchecked variables.	**High**
`cat` , `less` , `nano`	Show / page / edit a file	`nano hello.sh`	Low
`grep`	Search text	`grep -i error app.log`	None
`find`	Find files	`find /var/log -name "*.log"`	None (unless `-delete`)

`chmod` / `chown`	Change permissions / owner	`chmod 640 secret.env` (owner rw, group r, others none)	Medium. `chmod -R 777` is forbidden.
`systemctl`	Manage services	`systemctl status nginx` → `active (running)`	Medium
`journalctl`	Read system logs	`journalctl -u nginx --since "1 hour ago"`	None
`ps`, `top`	Processes, live CPU/RAM	`ps aux | grep nginx`	None
`df -h`, `free -h`	Disk and memory usage	`/dev/sda1 20G 6G 14G 30% /`	None
`ip a`, `ss -tulpn`	Addresses; listening ports	`LISTEN 0 511 0.0.0.0:80`	None
`curl`	Make web requests	`curl -I http://localhost` → `HTTP/1.1 200 OK`	None
`ssh`	Remote login	`ssh -i ~/.ssh/id_ed25519 user@203.0.113.10`	Medium

Core exercises

1. Create user `deploy`, add to group `www-data`, prove its permissions with `id`.
2. Install Nginx: `sudo apt update && sudo apt install -y nginx`; confirm `systemctl status nginx` and `curl localhost`.
3. Enable the firewall **after** allowing SSH, so you do not lock yourself out:

```
sudo ufw allow OpenSSH
sudo ufw allow 80,443/tcp
sudo ufw enable
sudo ufw status verbose
```

4. Write a cron job that records disk usage daily: `0 2 * * * /usr/local/bin/disk-report.sh`.
5. Harden SSH: key-only login, no root login (`PasswordAuthentication no`, `PermitRootLogin no`). Test a **second** session before closing the first.

Hardening baseline (checklist)

- Individual accounts + sudo; no shared logins
- SSH keys, MFA on cloud console, fail2ban or equivalent
- Unattended security updates; patch schedule
- Firewall default-deny inbound
- Time sync, log retention, disk-space alerts

Common mistakes: enabling firewall before allowing SSH; editing `sshd_config` without a backup and second session; running everything as root.

Assessment: practical — provision a clean Ubuntu VM, harden it and host a static page in 60 minutes unaided.

Level 3 — Networking Fundamentals (about 2 weeks)

Concept	Plain explanation	Example
TCP/IP, IPv4/IPv6	The rules for sending data; IPv4 has 32-bit addresses, IPv6 128-bit.	192.168.1.5 vs 2001:db8::1
Public vs private IPs	Private ranges (10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16) are not routable on the internet.	Database at 10.0.2.15
Subnets & CIDR	`/24` = first 24 bits are network; leaves 256 addresses (cloud reserves some).	10.0.1.0/24 = 10.0.1.0–10.0.1.255
Routing, NAT	A route table decides where traffic goes; NAT lets private hosts reach out via one public IP.	Private server updates through NAT Gateway
DNS records	A = name→IPv4; AAAA = IPv6; CNAME = alias; MX = mail; TXT = text/verification.	`www CNAME example.com`
DHCP	Automatically hands out IPs.	Home Wi-Fi
TLS certificates	Prove identity and encrypt traffic; they expire and must be renewed.	Let's Encrypt, ACM
VPN, reverse proxy, load balancer, segmentation	Encrypted private tunnel; a front door that forwards to apps; spreading traffic; splitting networks into zones.	Public ALB → private app subnet → private DB subnet

Practical exercises

1. Build a small local network (two VMs, one subnet) and ping between them.
2. Trace DNS: `dig +trace example.com`; explain each hop.
3. Connect to a remote Linux server by SSH key.
4. Configure a firewall that allows only 22 (your IP), 80, 443.
5. Diagnose a failed connection with this ladder: `ping` (reachable?) → `dig` (name resolves?) → `nc -vz host 443` (port open?) → firewall / security group → service listening (`ss -tulpn`) → app logs.
6. Serve a test app over HTTPS with Nginx and a certificate; verify with `curl -vI https://host`.

Common mistakes: overlapping CIDR ranges that block later VPC peering; forgetting DNS TTL delays; putting a database in a public subnet.

Level 4 — Programming and Scripting (about 3 weeks)

Minimum viable knowledge, taught through small projects, not theory alone.

Topic	Mini-project
Python fundamentals, error handling, logging	Script that checks a list of URLs, logs status and exits non-zero on failure.

Bash scripting	Backup script with `set -euo pipefail`, dated archive, retention clean-up.
YAML & JSON	Convert a config between formats; spot indentation errors.
Environment variables & secrets	Read a DB URL from the environment; confirm it is never printed to logs.
REST APIs	Call a public API with `curl` and Python `requests`; handle 4xx/5xx.
Basic SQL	`SELECT`, `INSERT`, `UPDATE`, `JOIN` on a sample database.
Regular expressions	Extract error codes from a log file.
Automation scripts	Nightly report of disk usage emailed or posted to a channel.

```
#!/usr/bin/env bash
# backup.sh - dated archive with 7-day retention. Test on a copy first.
set -euo pipefail
SRC="/var/www/app"; DEST="/backups"; STAMP=$(date +%F)
mkdir -p "$DEST"
tar -czf "$DEST/app-$STAMP.tar.gz" "$SRC"
find "$DEST" -name 'app-*.tar.gz' -mtime +7 -print -delete
echo "backup ok: $DEST/app-$STAMP.tar.gz"
```

Common mistakes: hard-coding secrets; ignoring exit codes; unquoted variables in `rm` / `find -delete`; no logging.

3. Cloud Platform Engineering (AWS First)

Teaching order: master AWS first (Level 5, about 4 weeks). Introduce Azure and GCP afterwards through *comparative exercises* ("build the same 3-tier app on Azure; what is the equivalent of a Security Group?"). Do not teach three clouds at once.

The 10-question template for every cloud service

For each service the trainee writes a one-page sheet answering: **1** What is it? **2** Why does it exist? **3** When to use? **4** When not to use? **5** How is it priced? **6** How to configure? **7** How to secure? **8** How to monitor? **9** How to troubleshoot? **10** What are the alternatives?

Service map and equivalents

Capability	AWS (primary)	Azure	GCP
Identity & access	IAM, Identity Center	Entra ID, RBAC	Cloud IAM
Virtual machines	EC2	Virtual Machines	Compute Engine
Private network	VPC, subnets, route tables, IGW, NAT GW	Virtual Network	VPC
Instance firewall	Security Groups / NACLs	Network Security Groups	VPC firewall rules
Object storage	S3	Blob Storage	Cloud Storage
Managed SQL	RDS	Azure SQL / Database for PostgreSQL	Cloud SQL
Load balancing	ALB / NLB	Load Balancer, Application Gateway	Cloud Load Balancing
Scaling	Auto Scaling Groups	VM Scale Sets	Managed Instance Groups
DNS & certificates	Route 53, ACM	Azure DNS, App Service certs / Key Vault	Cloud DNS, Certificate Manager
Containers	ECR, ECS, EKS	ACR, AKS	Artifact Registry, GKE
Monitoring & audit	CloudWatch, CloudTrail	Azure Monitor, Activity Log	Cloud Monitoring/Logging, Audit Logs
Secrets	Secrets Manager, SSM Parameter Store	Key Vault	Secret Manager
Backup	AWS Backup	Azure Backup	Backup and DR / snapshots

Ops & CI	Systems Manager, CodePipeline	Azure DevOps	Cloud Build

Worked example: reference 3-tier AWS architecture

Users → Route 53 DNS → ALB + ACM cert (public subnets) → App on EC2/ECS (private subnets) → RDS (isolated subnets)

- **Network:** one VPC, two Availability Zones, public / private-app / private-data subnets, NAT Gateway for outbound updates, S3 gateway endpoint.
- **Security:** ALB SG allows 443 from the internet; app SG allows only from ALB SG; DB SG allows 5432 only from app SG. No public DB IP.
- **Data:** RDS encrypted at rest, automated backups, Multi-AZ if the availability target requires it.
- **Ops:** CloudWatch alarms, CloudTrail enabled, AWS Backup plan, budget alert.

> **COST AWARENESS**
> NAT Gateways, load balancers, Multi-AZ databases, data transfer out and idle resources are common surprise costs. Always use the provider's official pricing calculator and record region, date and assumptions (Stage 05).

Per-service quick troubleshooting

Symptom	First checks
Cannot SSH to EC2	Security group port 22 from your IP; subnet route to IGW; public IP; key pair; instance status checks
ALB returns 502/503/504	Target health; app port; health-check path; SG from ALB to app; app logs
App cannot reach DB	DB SG source; subnet routing; DB endpoint; credentials from secret store
S3 AccessDenied	IAM policy, bucket policy, block-public-access, KMS key permission
Unexpected bill	Cost Explorer by service; look for NAT, data transfer, forgotten instances, unattached volumes, snapshots

4. DevOps Toolchain

Teach tools in the order a project needs them. **Essential** tools are required of every employee; **optional** tools are learned when a client needs them. **Do not introduce Kubernetes (or any tool) where a simpler method is sufficient.**

Category	Essential	Optional / situational	Primary use
Version control	Git, GitHub	—	Code, IaC, docs, reviews
IDE	VS Code	—	Editing, extensions for Terraform/Docker/YAML
CI/CD	GitHub Actions	GitLab CI/CD, Jenkins	Build, test, scan, deploy
Containers	Docker, Docker Compose	—	Packaging apps
Orchestration	—	Kubernetes (justified cases only)	Many services, scaling, self-healing
IaC	Terraform *or* OpenTofu	The other one	Repeatable infrastructure
Config mgmt	—	Ansible	VM configuration, patching
Reverse proxy	Nginx	—	TLS termination, routing
Monitoring	CloudWatch / native monitor	Prometheus, Grafana	Metrics, dashboards, alerts
Logging	Native cloud logging	Loki, OpenSearch	Central logs
Security scanning	Trivy, Gitleaks	Semgrep	Images/dependencies, secrets, code
Testing	Pytest	Postman	Code and API tests
Documentation	Markdown	Draw.io	Runbooks, diagrams
Project mgmt	GitHub Projects	Jira	Tasks and milestones

Tool pack (what every tool lesson must contain)

Installation → setup → configuration → commands → folder structure → use cases → assignment → troubleshooting. Examples of the first and most important:

Git & GitHub

```
git config --global user.name "Your Name"
git config --global user.email "you@company.com"
git switch -c feature/add-health-check
git add . && git commit -m "feat: add /health endpoint"
git push -u origin feature/add-health-check
```

Then open a Pull Request. **Troubleshoot:** merge conflicts, wrong branch, committed a secret (rotate it immediately; history rewrite is not enough).

Docker

```
docker build -t app:1.0 .
docker run -d -p 8080:8080 --name app app:1.0
docker logs app
docker ps
```

Troubleshoot: container exits (check `docker logs`); port already in use; no space (`docker system df`).

Terraform / OpenTofu

```
terraform fmt -recursive
terraform init
terraform validate
terraform plan -out=tfplan
terraform apply tfplan
```

Troubleshoot: state lock, drift, provider auth errors. Always read the plan.

Security scanners

```
gitleaks detect --source .
trivy fs .
trivy image app:1.0
```

Fail the pipeline on critical findings; triage the rest with owners and dates.

When is Kubernetes justified?

Option	Choose when	Operational effort
Single VM + Docker Compose	One app, low traffic, small team	Low
Managed PaaS / container service (App Service, Cloud Run, ECS Fargate)	Containers without cluster management	Low–Medium
Auto-scaling VMs behind a load balancer	Stateless app needing HA	Medium
Kubernetes (EKS/AKS/GKE)	Many services, frequent releases, team with K8s skills, need for portability	**High**

5. Client Project Lifecycle: Stages 01–18

Every client project follows these 18 stages in order. Each stage lists its objective, steps, deliverables, **acceptance criteria** and **approval gate**. A stage is closed only when the gate is signed by the named approver.

01 Lead → 02 Discovery → 03 Scope/SOW → 04 Architecture → 05 Cost → 06 Dev env → 07 Repo → 08 IaC → 09 Docker → 10 CI/CD → 11 K8s* → 12 Security → 13 Monitoring → 14 Backup/DR → 15 QA → 16 Production → 17 Handover → 18 Managed service

*Stage 11 only when Kubernetes is justified (see toolchain section).

Stage	Gate	Approver
01 Lead qualification	Proceed / decline	Sales or Project Manager
02 Discovery	Problem statement and requirements confirmed	Client
03 Scope freeze	SOW signed; commercial terms; access authorised	Client + Company
04 Architecture	Design approved	Technical Lead + Client
05 Cost	Estimate approved	Project Manager + commercial representative
08, 16 Infra change / Production	Plan reviewed, change approved	Technical Lead (independent of author) + Client for production
15 QA	Critical defects resolved or formally accepted	QA + Technical Lead
17 Handover	Written acceptance certificate	Client

Stage 01 — Lead Qualification

Objective: decide whether this client is a good fit before spending engineering time.

Collect: company name; industry; size; current application/infrastructure; business problem; expected outcome; technical requirements; estimated users and traffic; current hosting; budget range; timeline; security/compliance needs; decision-maker.

Deliverables: lead qualification document; discovery notes; opportunity assessment (fit score, risks, red flags such as “wants guaranteed 100% uptime on a tiny budget”).

> **GATE 1**
> Sales/Project Manager approves proceeding. Acceptance: all fields above filled or marked “unknown” with an owner.

Stage 02 — Client Discovery and Technical Assessment

Objective: understand reality before designing anything. Run a structured meeting; never assume.

Area	Questions to ask
Infrastructure	What runs where? Servers, versions, regions, who has access, any diagrams?
Application	Languages, frameworks, databases, third-party integrations, build and deploy steps today.
Traffic & availability	Users today / in 12 months, peak times, tolerated downtime per month.
Data	Sensitivity (personal, financial, health), location constraints, retention, encryption today.
Backup & recovery	What exists? Has restoration ever been tested? Acceptable data loss (RPO) and downtime (RTO)?
Cost	Current monthly cloud spend, invoices, known waste.
Limits	Legacy dependencies, licences, skills in the client team, policy restrictions.

Deliverables: discovery questionnaire (completed); current-state architecture diagram; infrastructure inventory; risk and dependency register.

> **GATE 2**
> Client confirms in writing the problem statement and requirements. Acceptance: inventory matches what the client can verify.

Stage 03 — Requirements and Scope Freeze (SOW)

Create a formal Statement of Work containing: objectives; in-scope and **out-of-scope** services; technical and infrastructure requirements; deliverables; responsibilities (ours vs client); timeline and milestones; dependencies; acceptance criteria; support period; change-request procedure.

> **DO NOT START IMPLEMENTATION**
> without (1) approved scope, (2) agreed commercial terms, (3) written access authorisation. Access is granted to named individuals with individual accounts, never shared admin credentials.

Change-request procedure

1. Client or team raises the change in writing.
2. Project Manager assesses impact on cost, time and risk.
3. Client approves the revised quote/timeline.
4. SOW addendum is signed before work begins.

Stage 04 — Infrastructure Architecture Design

Design ten views: application, cloud, network, database, storage, security, deployment, backup, monitoring, disaster recovery.

Create: system architecture diagram; network topology; component list; data-flow diagram; security-boundary diagram; deployment-flow diagram; **Architecture Decision Records (ADRs)**.

Compare at least two options

Criterion	Option A: single VM + Compose	Option B: ALB + auto-scaling + managed DB	Option C: Kubernetes
Cost	Lowest	Medium	Highest (cluster + people)
Security	Fewer controls	Good isolation	Powerful but larger attack surface
Complexity	Low	Medium	High
Scalability	Vertical only	Horizontal	Horizontal, fine-grained
Operational effort	Low	Medium	High

```
# ADR template (docs/adr/0001-choose-hosting.md)
Title: Use ALB + Auto Scaling + RDS
Status: Proposed | Accepted | Superseded
Context: why a decision is needed
Options considered: A, B, C with cost/security/complexity notes
Decision: B, because ...
Consequences: what becomes easier / harder
```

GATE 4

Technical Lead and client approve the architecture.

Stage 05 — Cloud Cost Estimation and Commercial Approval

Estimate **before** provisioning. Never invent prices; use the provider's official calculator and record **region, currency, pricing date, usage assumptions and taxes**. Estimates are not guaranteed bills.

Group	Items
A. One-time	Discovery, architecture, infrastructure setup, networking, migration, CI/CD, testing, documentation, training
B. Recurring	VMs, containers, Kubernetes, storage, databases, backups, bandwidth/data transfer, load balancers, DNS/certificates, monitoring, security tools, logging, support, third-party subscriptions
C. Internal	Developer hours, tech-lead review, project management, QA, support, contingency

```
Total Monthly Cloud Cost = Compute + Storage + Database + Network + Backup
                         + Monitoring + Security + Other services
Total Project Cost       = Labour + Setup + Migration + Testing + Tools + Other direct
costs
Selling Price            = Total Delivery Cost + Risk Allowance + Target Profit
```

Also calculate gross margin, contribution margin, break-even, MRR, ARR, expected cloud cost per customer, cost per active user. Provide scenarios at **100, 1,000, 10,000 and 100,000 users** where relevant (see worked template in the pricing section).

> **GATE 5**
> Project Manager and an authorised commercial representative approve the estimate.

Stage 06 — Corporate Development Environment Setup

Item	Requirement
Workstation	Company-approved, disk encryption, screen lock
Software	VS Code, Git, Docker, cloud CLI, Terraform/OpenTofu, Python or required runtime, Linux environment (WSL2 or VM)
Access	Individual GitHub account, SSH keys (ed25519, passphrase), password manager, **MFA on everything**
Workspaces	Project management board, documentation workspace
Environments	Separate **development, staging, production** (separate accounts/subscriptions/projects where possible)

> **FORBIDDEN**
> Shared root accounts, shared cloud admin credentials, shared personal access tokens, long-lived keys on laptops when SSO is available.

```
ssh-keygen -t ed25519 -C "you@company.com"      # create key (set a passphrase)
aws configure sso                                # preferred over static keys
aws sts get-caller-identity                      # verify who you are
```

Stage 07 — GitHub Repository and Folder Structure

```
client-project/
|-- README.md  CHANGELOG.md  LICENSE  .gitignore
|-- docs/                requirements, architecture, network design, deployment guide,
|   `-- runbooks/        backup policy, DR, security policy, troubleshooting
|-- infrastructure/
|   |-- terraform/{modules,environments/{dev,staging,production}}
|   `-- ansible/{inventories,playbooks,roles}
|-- application/{src,tests,Dockerfile,requirements.txt}
|-- deployment/{docker,kubernetes,nginx,scripts}
|-- ci-cd/github-actions/
|-- monitoring/{prometheus,grafana,alerts}
|-- security/{policies,scans,checklists}
`-- tests/{infrastructure,integration,performance,security}
```

Adapt to the project; **do not create empty folders for appearance**. Explain every folder to the trainee.

> **NEVER COMMIT**
> passwords, private keys, API secrets, cloud credentials, production env files, customer personal data, database dumps with sensitive data. Add them to `.gitignore` and scan with Gitleaks. If a secret is committed: **revoke and rotate it immediately**.

Repository rules: protected `main` branch; pull requests with at least one review; meaningful commit messages (`feat:`, `fix:`, `docs:`); release tags such as `v1.2.0`.

Stage 08 — Infrastructure as Code

What: describe infrastructure in text files so it is repeatable, reviewable and versioned. Process (follow in order):

1. Install Terraform or OpenTofu.
2. Authenticate securely (SSO / short-lived credentials).
3. Provider configuration.
4. Variables, 5. outputs, 6. reusable modules.
5. Remote state with locking (e.g. S3 + DynamoDB lock or native locking where supported).
6. `fmt` + `validate`.
7. Review the `plan`; 9. obtain approval; 10. `apply`; 11. verify resources; 12. document; 13. test controlled `destroy` in non-production.

```
provider "aws" { region = var.region }

variable "region"      { type = string  default = "ap-south-1" }
variable "environment" { type = string }

resource "aws_s3_bucket" "assets" {
  bucket = "client-${var.environment}-assets"
}
resource "aws_s3_bucket_public_access_block" "assets" {
  bucket                  = aws_s3_bucket.assets.id
  block_public_acls       = true
  block_public_policy     = true
  ignore_public_acls      = true
  restrict_public_buckets = true
}
output "assets_bucket" { value = aws_s3_bucket.assets.bucket }
```

> **PRODUCTION RULE**
> No production apply without peer review of the plan, a relevant backup, and explicit approval. Read every line marked `destroy` or `replace`.

Stage 09 — Docker and Containerisation

Concepts: container vs VM; image vs container; Dockerfile; Compose; volumes; networks; env vars; registries; multi-stage builds; image size; health checks; non-root users; image scanning.

```
# Dockerfile: multi-stage, non-root, health check
FROM python:3.12-slim AS build
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir --prefix=/install -r requirements.txt

FROM python:3.12-slim
RUN useradd -r -u 10001 appuser
WORKDIR /app
COPY --from=build /install /usr/local
COPY src/ ./src/
USER appuser
EXPOSE 8080
HEALTHCHECK --interval=30s --timeout=3s CMD python -c "import
urllib.request;urllib.request.urlopen('http://localhost:8080/health')"
CMD ["python","-m","src.main"]
```

```
# docker-compose.yml (development)
services:
  app:
    build: ./application
    ports: ["8080:8080"]
    environment:
      DATABASE_URL: ${DATABASE_URL}      # from untracked .env
    depends_on: [db]
    deploy: { resources: { limits: { cpus: "0.5", memory: 256M } } }
  db:
    image: postgres:16
    environment: { POSTGRES_PASSWORD: ${DB_PASSWORD} }
    volumes: [dbdata:/var/lib/postgresql/data]
volumes: { dbdata: {} }
```

Test: startup; `/health`; DB connection; `docker restart` (app recovers); data survives container removal (volume); failure handling (stop DB, app logs a clear error); resource limits; `trivy image` shows no unaccepted critical findings.

Stage 10 — CI/CD Pipeline Development

Commit → Pull request → Code review → Tests → Quality → Dependency scan → Secret scan → Image build → Image scan → Registry → Staging deploy → Integration tests → Approval → Production deploy → Smoke tests → Monitor

Each step exists to stop a specific class of failure: tests catch bugs, scans catch known vulnerabilities and leaked secrets, staging catches environment problems, approval catches business risk, smoke tests catch a broken release, rollback limits damage.

```
# .github/workflows/ci.yml
name: ci
on: { pull_request: { branches: [main] } }
permissions: { contents: read }
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: "3.12" }
      - run: pip install -r application/requirements.txt pytest
      - run: pytest application/tests
  security:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
        with: { fetch-depth: 0 }
      - uses: gitleaks/gitleaks-action@v2
      - run: |
          docker build -t app:${{ github.sha }} application
          docker run --rm -v /var/run/docker.sock:/var/run/docker.sock \
            aquasec/trivy image --exit-code 1 --severity CRITICAL app:${{ github.sha }}
```

```
# .github/workflows/cd.yml  (deploys only from main, production needs approval)
name: cd
on: { push: { branches: [main] } }
jobs:
  staging:
    runs-on: ubuntu-latest
    environment: staging
    steps: [ { uses: actions/checkout@v4 }, { run: ./deployment/scripts/deploy.sh
staging } ]
  production:
    needs: staging
    runs-on: ubuntu-latest
    environment: production        # configure required reviewers in GitHub
    steps: [ { uses: actions/checkout@v4 }, { run: ./deployment/scripts/deploy.sh
production } ]
```

Also deliver: environment-specific deployment; approval protection; **rollback procedure** (redeploy previous tagged image; database changes must be backward compatible); release notes; failure notifications to the team channel. Pin third-party actions to versions or commit SHAs and use OIDC rather than stored cloud keys.

> **NEVER**
>
> deploy to production straight from an unreviewed developer branch.

Stage 11 — Kubernetes Engineering (only if justified)

Teach only after containers and networking are solid. Topics: cluster architecture; nodes; pods; deployments; services; ConfigMaps; Secrets; ingress; persistent volumes; storage classes; namespaces; requests/limits; probes; autoscaling; RBAC; network policies; Helm; monitoring; upgrade and backup planning.

```
kubectl get nodes
kubectl apply -f deployment.yaml
kubectl get pods -n demo
kubectl describe pod <name> -n demo     # events show why it is Pending/CrashLoopBackOff
kubectl logs <name> -n demo
kubectl rollout undo deployment/app -n demo
```

Build a non-production demo cluster (kind/minikube or a small managed cluster) and then **cost it**: control-plane fees, nodes, load balancers, storage, plus engineer time to upgrade, patch and monitor. Present the comparison to the client; do not recommend Kubernetes by default.

Stage 12 — Security Engineering

Control	Minimum standard
Identity	MFA everywhere; individual accounts; least-privilege IAM; separate dev/prod permissions; no shared admin
Secrets	Secrets Manager / Key Vault / Secret Manager; never in Git or images
Encryption	TLS in transit; encryption at rest (disks, DB, buckets, backups)
Network	Default-deny firewalls; segmentation; DB in private subnets; SSH restricted or replaced by SSM/bastion
Vulnerabilities	OS patching; dependency, container-image and code scans in CI
Audit	CloudTrail/Activity Log/Audit Logs on; log retention; alerts on root/admin use
Backups	Encrypted, access-restricted, separate account/region where appropriate
Governance	Quarterly access reviews; incident response document; threat model before production

Frameworks to reference: OWASP, CIS Benchmarks, NIST Cybersecurity Framework, provider best-practice guides (e.g. AWS Well-Architected Security pillar).

Simple threat model (STRIDE) — do before production

1. Draw the data-flow diagram and trust boundaries.
2. For each flow ask: Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege.
3. Record threat, likelihood, impact, mitigation, owner.

> **AUTHORISATION**
> No penetration testing or intrusive scanning without written client authorisation and a defined scope. Scanning our own CI artefacts is fine; probing a client's live systems is not.

Stage 13 — Monitoring, Logging and SRE

Implement **before** launch. Monitor CPU, memory, disk, network, availability, response time, error rate, database performance, storage, cloud spend, certificate expiry, backup status, deployment status.

Term	Meaning	Example
SLI	A measurement	% of requests answered under 500 ms without error
SLO	Internal target for an SLI	99.5% per 30 days
SLA	Contractual promise with remedies	Set only after measuring real performance
Error budget	Allowed failure (100% − SLO)	99.5% → about 3.6 hours/month

Calculation check: 30 days = 43,200 minutes; 0.5% = 216 minutes = 3.6 hours. **Do not promise availability that the architecture and team cannot deliver**: a single VM cannot honestly back 99.99%.

Alert quality rules

- Alert on symptoms users feel (errors, latency, downtime) more than causes.
- Every alert has an owner, severity, runbook link and escalation path.
- Test alerts by triggering them. Remove noisy alerts.

```
# Prometheus alert example
- alert: HighErrorRate
  expr: sum(rate(http_requests_total{status=~"5.."}[5m])) /
sum(rate(http_requests_total[5m])) > 0.05
  for: 10m
  labels: { severity: page }
  annotations: { summary: "5xx rate above 5% for 10 minutes", runbook:
"docs/runbooks/high-5xx.md" }
```

Create: dashboards; alerts; escalation procedure; incident runbooks; monthly reliability report.

Stage 14 — Backup and Disaster Recovery

Define	Example answer
What	Database, uploaded files, configuration, IaC state, secrets metadata
Frequency / retention	DB: continuous + daily snapshot; keep 30 days; monthly keep 12 (agree with client)
Encryption / location	KMS-encrypted; second region or account for critical data
RPO	Maximum acceptable data loss, e.g. 15 minutes
RTO	Maximum acceptable downtime, e.g. 4 hours
Owner / test schedule	Named engineer; restoration test every quarter

> **RULE**
> A successful backup job proves nothing. Only a **tested restoration** proves recoverability. Record: backup used, steps, time taken (compare to RTO), data checked, issues found.

Stage 15 — Testing and Quality Assurance

#	Test area	Typical tool / evidence
1	Infrastructure validation	`terraform validate`, plan review, tflint
2	Configuration validation	yamllint, nginx `-t`, ansible-lint
3	Network connectivity	`curl`, `nc`, security group checks
4	Application functionality	Pytest, Postman collections
5	Database connectivity	App health endpoint, migration test
6	Integration	End-to-end flows in staging
7	Load / performance	k6, Locust: report p95 latency and error rate at target load
8	Security scanning	Trivy, Gitleaks, Semgrep reports
9	Backup restoration	Restore log + timings
10	Deployment	Pipeline run record
11	Rollback	Rollback executed and verified
12	Monitoring / alerts	Alert fired and received

QA report row template

ID	Objective	Preconditions	Steps	Expected	Actual	Pass/Fail	Evidence	Defect ref
QA-012	App survives DB restart	Staging up	1. Restart DB 2. Call /health	200 within 60 s			Screenshot/log	

All critical defects are resolved or **formally accepted** by the client in writing before release.

Stage 16 — Production Deployment

Pre-deployment verification	Done
Client approval of change plan	☐
Infrastructure matches approved IaC plan	☐
DNS and TLS certificates valid; renewal automated	☐
IAM permissions least-privilege; secrets present in secret store	☐
Database private, encrypted, backed up	☐
Monitoring, alerts, on-call contacts and cost alerts active	☐
Rollback plan tested	☐

Access restrictions confirmed (no open admin ports)	☐

Post-deployment: smoke tests; check logs; verify health and performance; confirm monitoring and first backup; record deployment evidence (time, version, approver, results). Deploy in a low-traffic window agreed with the client.

Stage 17 — Client Handover

Deliver: final architecture diagrams; infrastructure inventory; deployment and CI/CD documentation; access and permissions register; backup/recovery documentation; monitoring dashboards; incident runbooks; troubleshooting guide; cost breakdown; maintenance schedule; training session; **acceptance certificate**; support and escalation contacts.

CREDENTIALS

Transfer through the approved secrets manager or secure sharing with expiry. Never place passwords or private keys in handover documents. Remove our temporary access if the client is not on a managed plan.

GATE 17

Written client acceptance obtained and filed.

Stage 18 — Maintenance and Managed Services

Service	Cadence
Monitoring and alert response	24×7 or business hours, per contract
Security patching	Monthly; critical within an agreed window
Backup verification and DR test	Monthly check; quarterly restoration test
Certificate and renewal monitoring; CI/CD upkeep	Continuous / monthly
Cost optimisation and capacity planning	Monthly review
Incident management; monthly report	As needed; monthly

Contract must define support hours, response targets by severity, escalation, exclusions and service-level commitments. Avoid “unlimited” support or usage unless priced for it.

Severity	Definition	Example response target (set per contract)
P1	Production down / data at risk	Acknowledge fast, update frequently
P2	Major feature degraded	Same business day
P3	Minor issue, workaround exists	Within agreed business days

6. Roles and Project Management

Role	Responsibilities
Sales Executive	Lead generation, first contact, scheduling discovery, proposal coordination
Project Manager	Plan, timeline, resources, client communication, risk and change management
Cloud Engineer	Cloud architecture implementation, provisioning, networking, security configuration
DevOps Engineer	Git workflows, CI/CD, containers, IaC, deployment automation
QA Engineer	Test planning, infrastructure validation, performance testing, security verification
Technical Lead	Architecture approval, code and infrastructure review, production approval

In a startup one person may hold several roles, but **the author of a change must never be its only approver**. For critical production changes use independent review wherever possible.

RACI for key activities

Activity	Cloud/DevOps Eng.	QA	Tech Lead	PM	Client
Architecture	R	C	A	I	A
Infrastructure change	R	C	A	I	I
QA sign-off	C	R	A	I	I
Production release	R	C	A	C	A
Scope change	C	I	C	R	A

R = Responsible, A = Accountable/approves, C = Consulted, I = Informed.

7. Internal Operations Portal

One internal portal (build or buy) to run the division. Required modules:

People

- Employee management
- Training progress
- Task assignment

Sales & clients

- Lead management
- Client management
- Billing and recurring revenue

Delivery

- Project management and milestones
- Change requests
- QA reports
- Deployment history

Operations

- Infrastructure inventory
- Cloud cost tracking
- Incident management
- Support tickets, SLA tracking

Knowledge

- Documentation repository
- Monthly reports

Mandatory fields for every project record

Unique project ID · client ID · assigned employee · project manager · current stage · milestones · due dates · approval status · cost status · risk register · delivery status.

> **SECURITY**
> The portal must never store cloud root passwords, API keys or private keys. Link to the approved secrets manager instead. Protect the portal with SSO/MFA and audit logs.

8. Pricing and Profitability

Commercial model for every proposal

One-time project price	Monthly managed service
Discovery, architecture, development, cloud configuration, migration, CI/CD, security, QA, documentation, training, project management	Cloud resources, monitoring, maintenance, security reviews, backup verification, incident support, reporting, support hours, third-party subscriptions

Required financial outputs

Direct project cost · internal labour · third-party cost · cloud cost · contingency · total cost · selling price · gross profit · gross margin · MRR · ARR · break-even · renewal price · additional usage charges.

Formulas

```
Gross profit      = Revenue - Direct costs
Gross margin %    = Gross profit / Revenue x 100
Contribution      = Revenue - Variable costs
Break-even (units)= Fixed costs / (Price per unit - Variable cost per unit)
MRR               = Sum of monthly recurring fees
ARR               = MRR x 12
Cost per user     = Monthly cloud cost / Active users
```

Worked example (illustrative numbers only, not real prices)

Assumptions: labour 120 hours at an internal cost of 20 per hour; tools 100; contingency 15% of subtotal; target profit 30% of total cost. Currency units are generic.

Line	Calculation	Result
Labour	120 × 20	2,400
Tools and other direct		100
Subtotal	2,400 + 100	2,500
Contingency 15%	2,500 × 0.15	375
Total delivery cost	2,500 + 375	2,875
Target profit 30%	2,875 × 0.30	862.50
Selling price	2,875 + 862.50	**3,737.50**
Gross margin	862.50 / 3,737.50	23.1%

Note: a 30% markup on cost is a 23.1% margin, not 30%. To reach a 30% margin the price is cost / (1 − 0.30) = 2,875 / 0.70 = 4,107.14. Always state which one you mean.

Scenario table template (fill from the official calculator)

Users	Compute	DB	Storage	Network	Monitoring	Total / month	Cost per user
100	Fill from provider calculator: region, date, assumptions						
1,000							
10,000							
100,000							

Expense classification (put it in every contract)

Type	Meaning
Customer-billed cloud	Client's own cloud account, paid directly to the provider
Company-managed cloud	Our account; resold with a stated management fee
Pass-through third party	Licences/subscriptions billed at cost
Service fee	Our labour and expertise

Prefer the client owning their cloud account with us holding limited, individual access. Define additional-usage rates and caps; avoid unlimited support or usage unless priced in.

9. Client Acquisition and Deal Closing

Target customers: SaaS startups; software companies; healthcare; e-commerce; manufacturing; financial and professional services; growing SMEs; businesses on outdated infrastructure.

Channels: LinkedIn; Upwork; Clutch; direct email; technology partnerships; startup communities; referrals; existing web/app development clients.

15-step sales process

1. Identify potential customer.
2. Research their technology.
3. Find a real infrastructure problem.
4. Contact the decision-maker.
5. Schedule discovery.
6. Conduct technical assessment.
7. Prepare solution proposal.
8. Present architecture.
9. Explain implementation and recurring costs.
10. Offer a paid pilot where appropriate.
11. Agree scope and contract.
12. Obtain payment and authorisation.
13. Start delivery.
14. Demonstrate results.
15. Convert to a managed-services contract.

> **HONESTY RULE**
> Never promise savings, performance or security outcomes without supporting evidence. Say “we will measure and report,” not “you will save 40%.”

Template: cold email

```
Subject: Quick question about deployments at {Company}

Hi {Name},
I noticed {specific observation, e.g. your app runs on a single server / releases are
manual}.
We help {industry} teams set up reliable cloud hosting, automated deployments and
monitoring,
usually starting with a short, fixed-price assessment.
Would a 20-minute call next week be useful to see whether this applies to you?
- {Your name}, VEL Software Solutions
```

Template: LinkedIn outreach

```
Hi {Name}, I work with {industry} teams on cloud reliability and deployment automation.
Saw {company/news}. Happy to share a short checklist we use for pre-launch cloud
reviews, no strings attached.
```

Template: discovery questionnaire (short form)

- What is the business problem and what does success look like?
- Where does the application run today and who maintains it?
- How many users, and what happens if it is down for an hour?
- What data do you hold and are there regulatory duties?
- When was the last backup restored successfully?
- What is the monthly hosting bill, budget range and deadline?
- Who approves scope and payment?

Proposal and contract pack

Document	Must contain
Technical assessment	Findings, evidence, risks, quick wins
Proposal	Problem, solution, architecture summary, timeline, price, assumptions, exclusions
SOW	As in Stage 03
Pricing sheet	One-time, monthly, usage charges, validity period
Paid pilot agreement	Small fixed scope, fixed price, success criteria, what follows if it succeeds
Client presentation	Business-first story: problem → solution → risk → cost → next step
Handover certificate	List of deliverables, date, client signature

Objection handling: “Too expensive” → show cost of downtime and options at different price points. “We can do it ourselves” → offer assessment or training. “We are on Vendor X” → skills apply to any cloud. Legal terms are reviewed by a qualified lawyer before use.

10. Practical Training Projects and Sign-off

The employee must complete these projects, in order, in a personal sandbox account with a budget alert set, before working independently on client work. Each is reviewed by the Technical Lead against its acceptance criteria. Evidence (repo link, screenshots, logs) goes in the trainee's portfolio.

#	Project	Skills proven	Acceptance criteria
P1	Hardened Linux web server	Ubuntu, SSH, UFW, Nginx, systemd, TLS	Key-only SSH; only 22/80/443 open; HTTPS valid; service restarts after reboot; hardening checklist 100%
P2	Git workflow and secret hygiene	Branches, PRs, reviews, Gitleaks	Protected main; PR with review; secret scan passes; a deliberately committed fake secret is detected and the rotation steps documented
P3	Containerised app + database	Docker, Compose, volumes, health checks	Non-root image; data survives restart; Trivy has no unaccepted critical issues; resource limits set
P4	AWS 3-tier network with Terraform	VPC, subnets, SGs, ALB, RDS, IaC	DB not public; plan reviewed; remote state; clean `destroy` in sandbox; cost estimate with stated assumptions
P5	CI/CD pipeline	GitHub Actions, scanning, environments	PR runs tests + scans; staging auto-deploy; production needs approval; rollback demonstrated
P6	Monitoring and alerting	CloudWatch or Prometheus/Grafana, logs	Dashboard for CPU, memory, disk, errors, latency; an induced failure triggers an alert received by the trainee; runbook written
P7	Backup and disaster recovery drill	Snapshots, restore, RPO/RTO	Database restored to a new instance from backup; measured recovery time vs RTO documented
P8	Azure/GCP comparative build	Cloud equivalence	Same app deployed on a second cloud; comparison table of services, cost drivers and gotchas
P9	Capstone: mock client engagement	All 18 stages	Complete lifecycle for a fictional client: questionnaire, SOW, architecture + ADR, cost model, IaC, pipeline, security review, QA report, handover pack, and a 20-minute client presentation

Assessment and levels

Level	Title	Can do	Evidence
0	Trainee	Completes Levels 1–4 exercises	Quizzes + practical
1	Associate	P1–P5; works on tasks under supervision	Reviewed portfolio

2	Engineer	P1–P8; builds non-production environments; participates in reviews	Lead sign-off
3	Independent engineer	P9; may own a project stage; production changes still need independent approval	Capstone passed

Suggested timeline: weeks 1–2 Level 1; 3–5 Level 2; 6–7 Level 3; 8–10 Level 4; 11–14 AWS and P1–P4; 15–18 CI/CD, security, monitoring, P5–P7; 19–22 Azure/GCP and capstone. Adjust to the learner; competence matters more than speed.

Trainee code of conduct

- Ask early; stuck for 30 minutes means ask.
- Treat client data and credentials as sacred.
- Write down what you did: if it is not documented, it did not happen.
- Destroy sandbox resources after each lab to avoid cost.

11. Appendices

A. Production readiness checklist

Area	Check
Security	MFA, least privilege, no public DB, secrets in vault, encryption, scans clean, threat model done
Reliability	Health checks, auto-restart/scaling, tested rollback, load test passed
Data	Backups automated, restoration tested, RPO/RTO met
Observability	Dashboards, alerts tested, logs centralised, on-call contacts
Cost	Budget alerts, tagging, estimate vs actual reviewed
Operations	Runbooks, documentation, access register, change plan approved
Commercial	SOW accepted, support terms agreed, handover scheduled

B. Incident response (first 30 minutes)

1. **Acknowledge** the alert; assign an incident lead.
2. **Assess** severity and user impact; open an incident record.
3. **Communicate** to the client with what is known, what is being done, next update time.
4. **Mitigate** first (rollback, scale, failover); find root cause after service is restored.
5. **Record** timeline and actions.
6. **Post-incident review** within days: blameless, root cause, actions with owners.

C. Change request template

```
ID / Date / Requester
Description and reason
Systems affected / environments
Risk and impact assessment
Implementation steps
Test steps
Rollback steps
Backup taken? (Y/N + reference)
Approvers: Technical Lead ____  Client ____
Result and evidence
```

D. Monthly managed-service report outline

- Availability vs SLO; incidents and root causes
- Patching and vulnerability status
- Backup success and last restoration test
- Cost vs budget with optimisation recommendations

- Capacity trends; planned work for next month

E. Glossary

Term	Meaning
IaC	Infrastructure as Code: infrastructure defined in version-controlled files
CI/CD	Continuous Integration / Delivery: automated build, test, deploy
VPC	Virtual Private Cloud: your isolated network in the cloud
IAM	Identity and Access Management
RPO / RTO	Maximum data loss / maximum downtime acceptable
SLI / SLO / SLA	Measurement / internal target / contractual promise
ADR	Architecture Decision Record
SOW	Statement of Work
MFA	Multi-factor authentication
Least privilege	Grant only the permissions needed, no more

F. Official references to consult (verify current versions)

- AWS Well-Architected Framework and AWS Pricing Calculator; Azure Architecture Center and Pricing Calculator; Google Cloud Architecture Framework and Pricing Calculator
- OWASP Top 10; CIS Benchmarks; NIST Cybersecurity Framework
- Terraform / OpenTofu, Docker, Kubernetes and GitHub Actions official documentation
- Google SRE Book (free online)

> **DOCUMENT CONTROL**
> Document VEL-CD-001 · Version 1.0 (draft for management approval) · Owner: V. LOKESHWARAN · Review: quarterly or after any failed project · Contact: velsoftwaresolutions.in@gmail.com. Prices and figures in examples are placeholders until verified from official sources.